

Corso di Laurea in Ingegneria e Scienze Informatiche

Progettazione di un configuratore di vendite

Tesi di laurea in:
BASI DI DATI

Relatore

Prof.sa Annalisa Franco

Candidato

Cavina Mattia

Sommario

Max 2000 characters, strict.

Indice

Sommario	iii
1 Introduzione	1
2 Situazione As-Is	3
2.1 Premessa	3
2.2 Situazione infrastrutturale aziendale	3
2.3 Attuali software utilizzati	4
2.4 Organizzazione aziendale e flussi di lavoro	5
3 Raccolta dei requisiti	7
3.1 Scelta dei candidati	7
3.2 Modalità di svolgimento raccolta informazioni	8
3.3 Riassunto punti emersi	9
3.3.1 Rispetto all'attuale sistema	9
3.3.2 Richieste avanzate	10
4 Analisi dei requisiti	13
4.1 Requisiti funzionali	13
4.2 Requisiti non funzionali	15
5 Progettazione dell'architettura	17
5.1 Preambolo sulle scelte architetturali	17
5.2 Deployment	17
5.3 Package	18
5.3.1 Interfaccia al database	19
5.3.2 Libreria di componenti View	19
5.3.3 Modulo Main Configuratore	19
5.4 Flussi di lavoro	20

6	Scelte progettuali	23
6.1	Scelta modello di sviluppo	23
6.2	Suddivisione e contenuto dei rilasci	24
6.3	Scelta Framework ORM	25
6.4	Scelta Framework grafico	26
6.5	Scelta del pattern di progettazione	26
7	Progettazione del database	29
7.1	Esportazione Vecchio Db	30
7.2	Creazione regole di denominazione	30
7.3	Utilizzo di SSIS nel processo di migrazione	30
7.4	Ristrutturazione database	30
7.4.1	Eliminazione ridondanze e normalizzazione	30
7.4.2	Creazione tabelle di lookup	30
7.4.3	Aggiunta tabelle non presenti	30
7.4.4	Creazione tabelle di Back Up	30
7.4.5	Scelta di strutture Avanzate come Triggers e Stored Procedure	30
7.5	Schema logico finale	30
7.6	Popolamento database	30
7.7	Dati fissi	30
7.8	Dati fissi da vagliare	30
7.9	Dati provenienti da altri DB	30
7.9.1	Scelta dei dati	30
7.9.2	Query di sincronizzazione e tempistiche di aggiornamento	30
8	Creazione Model	31
8.1	Mapping delle classi	31
8.2	Struttura del package	31
8.2.1	Classi di Test	31
8.2.2	Repository	31
8.2.3	Classi ed Interfacce esposte	31
9	Creazione View	33
9.1	Utilizzo di Figma per disegno interfacce	33
9.2	Creazione delle Factory per le View	33
9.3	Scelta delle classi di viewmodel	33
10	Modulo controller	35
10.1	Utilizzo della Dependency Injection	35
10.1.1	Utilizzo di multi Threading per la gestione dei dati	35
10.2	Scelta gestione transazione e dei commit	35

10.3 Utilizzo della Cache in memoria per la gestione di dati fissi	35
11 Modulo configurazione prodotto	37
11.1 Scopo del modulo	37
11.2 Focus group individuazione metodologia	37
11.3 Studio sulla guida alla configurazione	37
11.4 Query utilizzate per la ricerca	37
	39
Bibliografia	39

Elenco delle figure

2.1	Schema degli stati di una offerta commerciale	6
4.1	Requisiti funzionali raccolti.	14
4.2	Requisiti non funzionali raccolti.	15
5.1	Diagramma dei deployment	18
5.2	Diagramma dei package	20
5.3	Nuovo diagramma degli stati di un contratto	21
6.1	Diagramma funzionamento MVVM	27

Capitolo 1

Introduzione

Il progetto di tesi è volto alla progettazione ex-novo del programma di vendita dell'azienda Cepi S.p.a. , azienda con una produzione di tipo Engineering To Order (ETO) specializzato nella progettazione di impianti di stoccaggio custom. La tipologia di produzione ETO ha portato ad un profondo studio della situazione odierna, delle sue esigenze attuali e future tenendo aperte le possibilità di ampliamenti. L'analisi ha comportato un sostanziale lavoro di programmazione e raccolta dati per coprire nel modo più ampio i bisogni aziendali. Questa prima fase è stata condotta inizialmente attraverso la selezione di figure chiave scelte per mansione o/e ruolo, che potessero dare una visione dei bisogni del proprio o altrui reparto e successivamente sottoponendoli ad interviste. Sono state condotte 14 interviste, di cui 8 in modalità vis a vis, mentre le restanti 6 attraverso la dimostrazione dell'attuale applicativo utilizzato. Il risultato delle interviste condotte, ha fatto emergere attuali e nuove necessità architetture da affrontare come il cross-platform, l'utilizzo offline del database e la necessità di costruire offerte in modo più agile e strutturato. Questa prima fase ha inoltre sottolineato come il progetto non debba solo sostituire l'attuale sistema, ma debba inserirsi in un contesto di rinnovamento più ampio, coordinandosi con altri programmi aziendali e relativi database in ottica "single-principal responsibility (SRP)" eleggendo dei master data che siano i soli responsabili della creazione di certi dati. Il SRP ha plasmato notevolmente la costruzione del database ricreando, dove possibile, le strutture pre esistenti da cui si necessitava recuperare i dati mediante reverse

engineering.section 7.9 L'intero sistema è stato pensato per appoggiarsi all'infrastruttura esistente aziendale, come SQLServer[Micd], ma studiando delle logiche di flessibilità e ampliamento dinamico studiando accuratamente le relazioni tra le tabelle. Si è reso necessario cercare e scegliere inoltre un framework per la parte grafica che potesse soddisfare il cross-platform

essendo esso nelle richieste avanzate. La costruzione delle interfacce grafiche, è stata supportata dal sito "Figma"[Fig] che ha permesso una progettazione in ottica mobile first.9 La scelta accurata dei framework e le strutture dati nella fase iniziale, ha pesato molto in termini di tempo e progettazione, per garantire al richiedente un sistema che non rischiasse né il debito tecnico, né la preclusione a sviluppi futuri. La pianificazione dei primi test con gli utenti, dei rilasci delle demo e del metodo di progettazione sono state cruciali per dettare tempi e priorità degli item dovendo coprire in modo esaustivo almeno il core del progetto.

Capitolo 2

Situazione As-Is

2.1 Premessa

La situazione infrastrutturale aziendale è stata studiata dal sottoscritto nel corso del tirocinio formativo. La raccolta dei dati è stata svolta mediante interviste e studio dell'attuale sistema informatico. Nelle sezioni che seguono introdurrò brevemente l'As-Is.

2.2 Situazione infrastrutturale aziendale

L'infrastruttura informatica aziendale si appoggia su server fisici allocati internamente e gestiti mediante Active Directory. I server fisici sono, a loro volta virtualizzati per gestire separatamente le memorie in base alla finalità garantendo isolamento delle informazioni e permessi di accesso. Ogni reparto ha accesso a determinati server virtuali in cui gestisce in autonomia i propri contenuti. I database dei principali programmi sono gestiti all'interno di un server dedicato dove è installato un motore SQL Server [Mied]. I principali database aziendali sono:

- **AdHoc[Zuc]**: l'enterprise resource planning (ERP) aziendale, programma che gestisce le attività aziendali come gli approvvigionamenti e la movimentazione dei prodotti, le vendite, le fatturazioni nonché i cicli produttivi.

- **Fusion[Rul]**: il product lifecycle management (PLM) aziendale, programma volto alla gestione dei prodotti meccanici, dalla loro creazione all'archiviazione dei disegni con relative distinte dei componenti. Altre funzioni applicate sui prodotti sono la gestione delle revisioni o versionamenti, ed infine la loro fine produzione.
- **BombConfine**: database ponte tra i database esterni e quello dell'ERP. All'interno sono presenti tabelle di frontiera scritte da programmi esterni e che il gestionale usa per importare i dati.

La sicurezza aziendale è sostenuta da network address translation (NAT) e Firewall che impediscono l'accesso dall'esterno se non tramite virtual private network (VPN).

2.3 Attuali software utilizzati

Nel capitolo precedente abbiamo fatto cenno ai database ed ai relativi programmi che gestiscono quanto concerne l'ERP e il PLM. Per quanto riguarda le vendite, punto focale dell'intervento richiesto, l'azienda utilizza attualmente un'applicazione sviluppata internamente. L'attuale sistema sviluppato in Access, programma della suite Microsoft Office, comprende al suo interno sia il frontend che il backend. Il software è stato implementato circa 30 anni fa ed ha subito numerosi aggiornamenti e modifiche nel tempo per adattarsi alle esigenze commerciali e all'evoluzione del mercato. L'applicazione consente di creare, modificare e gestire i contratti con i clienti, includendo funzionalità quali la gestione degli sconti, la personalizzazione delle offerte e la stampa dei documenti. Tuttavia, nonostante le funzionalità implementate, il software presenta significative criticità che ne limitano l'efficienza e l'usabilità, come confermato dalle interviste condotte con gli utenti. Tali criticità includono instabilità operativa, mancanza di standardizzazione, codifiche obsolete, difficoltà negli aggiornamenti di prezzo e assenza di integrazione con altri sistemi aziendali. Di conseguenza, è indispensabile sviluppare un nuovo programma che affronti tali problematiche e migliori l'esperienza degli utenti, preservando nel contempo le funzionalità essenziali del sistema attuale. Il programma Access è stato fondamentale in questi anni per l'utilizzo in zone offline. Questa funzionalità è

stata favorita dalla natura del programma stesso, il quale racchiude in un solo file database e frontend, copiandolo in locale e sincronizzandolo una volta tornato in sede.

2.4 Organizzazione aziendale e flussi di lavoro

Il flusso lavorativo è risultato non essere un aspetto secondario nello sviluppo richiesto. Esso si deve interporre tra il reparto commerciale, ovvero addetti alle vendite, i reparti progettuali e produttivi come uffici tecnici, logistica ed amministrazione. L'attuale flusso di lavoro analizzato pone il reparto commerciale come punto di partenza dell'intero flusso che arriva fino al post-vendita. Il contratto, creato da tale reparto, dopo essere stato stipulato viene inviato al reparto amministrativo che ne importa i dati all'interno del gestionale. Simultaneamente esso viene mandato al reparto project manager che lo revisiona tecnicamente in caso di errori progettuali. Dopo la revisione tecnica, il contratto viene spedito al reparto logistico che preventivamente verifica i dati di destinazione ed organizza le spedizioni sulla base del venduto. A causa del sistema obsoleto utilizzato ed il debito tecnico accumulato negli anni, l'attuale sistema è isolato rispetto ai programmi utilizzati dall'azienda. Quest'ultimo punto fa parte delle richieste imprescindibili da risolvere. L'integrazione nell'ecosistema informatico passa prima di tutto dall'unificazione dei codici, ovvero quelli a listino con i prodotti a gestionale, ma anche creando un flusso di notifiche tra i vari ambiti.

Capitolo 3

Raccolta dei requisiti

3.1 Scelta dei candidati

La raccolta requisiti si è svolta mediante la selezione di figure aziendali con distinti ruoli ed esperienza. Queste figure selezionate sono state poi sottoposte ad una intervista sia in modalità vis a vis, che in alcuni casi da remoto per favorire la registrazione del meeting. I candidati possono essere raggruppati in soci aziendali, utilizzatori, e non utilizzatori. All'interno degli ultimi due gruppi sono state selezionate, per far fronte ai basias dovuti all'abitudine, candidati sia con esperienza pluriennale che neoassunti. In totale sono state intervistate 14 persone così suddivise:

Utilizzatori abituali:

- **Agenti commerciali esterni:** 2 persone
 - 1 agente con esperienza limitata.
 - 1 agente con esperienza consolidata.
- **Commerciali interni:** 4 persone
 - 1 specializzato in nuovi impianti.
 - 1 specializzato in ricambi.

- 2 con competenze trasversali sia per quanto concerne nuovi impianti che post-vendita.

Utilizzatori saltuari:

- **Soci aziendali con responsabilità:** 3 persone
 - 1 responsabile delle vendite.
 - 1 responsabile tecnico.
 - 1 responsabile amministrativo.
- **Direttore di produzione:** 1 persona

Non utilizzatori:

- **Impiegati tecnici:** 4 persone
 - 1 Project manager.
 - 1 Responsabile logistica.
 - 1 Progettista meccanico.
 - 1 Progettista elettrico.

L'ordine delle interviste svolte non è stato lineare reparto per reparto, ma si è optato di partire prima di tutto dalla proprietà per capire la direzione ed aspettative del progetto. Le restanti interviste hanno seguito un ordine sparso alternando utilizzatori e non utilizzatori, cercando di non saturare subito ogni reparto in modo da sfruttare quanto appreso dalle interviste precedenti per guidare le successive.

3.2 Modalità di svolgimento raccolta informazioni

Le interviste, come detto precedentemente, si sono svolte sia in presenza che da remoto, ma sempre singolarmente. Con permesso degli interlocutori, esse sono state registrate, anche solo l'audio, per poi essere rielaborate e trascritte. La

scelta di utilizzare la modalità fisica o da remoto è stata dettata dalla necessità, per utilizzatori abituali, di registrare visivamente come utilizzano l'attuale applicativo. Tramite questa modalità si è potuto captare i flussi di utilizzo e le principali funzioni presenti da mantenere. Nei restanti incontri si è preferito un approccio in presenza con l'interlocutore, facilitato la raccolta di requisiti e consigli sul progetto.

Le prime interviste sono state supportate da una scaletta di domande, preparate sulla base dell' AS-IS² che hanno aiutato a guidare l'incontro. Con il progredire delle interviste è stato adottato un approccio più flessibile, portando i risultati delle interviste precedenti alle successive. In questo modo si potuto distillare quali bisogni erano i più condivisi. Questo approccio ha consentito inoltre di ordinare i requisiti per importanza.

3.3 Riassunto punti emersi

3.3.1 Rispetto all'attuale sistema

Funzionalità da mantenere

Nelle interviste raccolte sono stati sottolineati una serie di funzionalità da mantenere. Tali funzionalità sono:

- Gestione delle sconti a cascata. Attualmente si ha la possibilità di inserirle in più punti, ovvero nella singola voce, nel capitolo, o dell'offerta stessa.
- Gestire all'interno delle voci o capitoli opzionali che non concorrono al prezzo offerto, ma ad uno opzionale.
- Gestione di porzioni di contratto raggruppati in capitoli.
- I campi del database che attualmente concorrono ai report, sono da mantenere.
- Creazione reportistica formato PDF del contratto digitale. Da gestire, come ora, il multilingua del report stesso.
- Esportazione all'ERP delle voci contrattuali presenti nell'ordine.

Criticità emerse

Mediante le dimostrazioni di utilizzo si è potuto raccogliere le criticità da attenzionare, come:

- Blocchi frequenti in condizione di concorrenza. Se più utenti utilizzano in contemporanea il file si generano deadlock.
- Non è presente una gestione centralizzata dei listini ed l'aggiornamento dei codici, attualmente si gestisce esternamente tramite file Excel.
- Mancanza di linee guida nella compilazione delle offerte. Si è avanzata l'ipotesi di un configuratore di prodotto che aiuti l'utente.
- Mancanza di gestione dei valori duplicati e controlli sulle chiavi porta a creare clienti e prodotti duplicati.

3.3.2 Richieste avanzate

Gestione ruoli e permessi

La scelta di coprire quanti più reparti possibili ha fatto emergere la mancanza di gestione dei ruoli. Si è richiesto quindi di attribuire ad ogni utente un ruolo, a sua volta ognuno di essi avrà viste e permessi dedicati. I ruoli individuati sono:

- Admin con visione sull'intero sistema, con possibilità esclusiva di modifica ad alcune tabelle fondamentali.
- Commerciali interni che possono gestire nell'interesse clienti e contratti.
- Commerciali esterni all'azienda, esempio agenti a partita IVA, possono vedere solo i propri clienti e contratti.
- Amministrazione contabile, può modificare i dati contabili dei clienti. Questo ruolo può altresì esportare le voci contrattuali all'ERP per gestire la fiscalità.
- Project Manager con compito di revisionare tecnicamente il contratto. Questo compito è volto alla gestione a monte dell'invenduto.
- Visualizzatore senza permessi di scrittura.

Gestione dei contratti

Nella creazione dei contratti, core del progetto, si dovrà sviluppare il vero salto di qualità. Nelle interviste si è cercato di raccogliere richieste che, anche dai primi rilasci, potessero dare al richiedente un sistema più ottimale. Tali richieste sono:

- Gestione storico dei dati in caso di modifica.
- Gestione di più versioni di uno stesso contratto legati dal medesimo codice progressivo. In caso di approvazione serve far decadere le versioni non approvati.
- Importazione dal PLM di dati tecnici dei prodotti. Questi dati sono utili nei calcoli utili alla contrattazione (Es: Potenza elettrica totale consumata).
- Gestione degli stati di avanzamento di un progetto e relative notifiche.
- Revisione tecnica del venduto in modo da dare la possibilità di proporre all'occorrenza integrazioni contrattuali.

Capitolo 4

Analisi dei requisiti

4.1 Requisiti funzionali

In questa sezione riassumo tutti i requisiti funzionali emersi. Essi sono raggruppati per area di competenza all'interno dello sviluppo.

4.1. REQUISITI FUNZIONALI

Area / Modulo	Requisito Funzionale	Descrizione / Specifiche	Ruoli Coinvolti
Sicurezza & Accessi	Profilazione RBAC granulare	Gestione permessi per tabella (lettura/scrittura) tramite tabelle utenti e utenti_tipi.	Tutti i ruoli
Anagrafica Clienti	Controllo anti-duplicazione	Validazione obbligatoria dell'univocità tramite Partita IVA in fase di inserimento.	Amministrazione, Agenti, Commerciali
Anagrafica Clienti	Sedi multiple e tipizzazione	Gestione sedi legali/operative, destinazioni merce separate e distinzione cliente finale / rivenditore.	Amministrazione, Agenti
Anagrafica Clienti	Storico contratti cliente	Consultazione integrata di tutte le offerte e ordini emessi per lo specifico cliente.	Commerciali, Amministrazione
Catalogo Prodotti	Codifica univoca	Allineamento del codice prodotto tra Configuratore, Ad-Hoc ERP e Fusion PLM.	Admin, Commerciali
Catalogo Prodotti	Gestione obsoleti e visibilità	Blocco dei componenti obsoleti rilevati da PLM; flag di visibilità per migrazione progressiva da ERP.	Admin, PM / Engineering
Catalogo Prodotti	Selezione per Macrogruppi	Navigazione guidata ad albero (Macchina → Componenti → Accessori → Ricambi) con vincoli di compatibilità.	Commerciali Nuovi/Ricambi
Catalogo Prodotti	Scheda caratteristiche tecniche	Attributi tabellari strutturati: meccanici, elettrici, pneumatici, logistici e accessori correlati.	Commerciali, PM / Engineering
Catalogo Prodotti	Descrizioni standardizzate	Modello ibrido: voce principale da menu standardizzato (da Fusion) affiancata da campo note libero.	Commerciali, PM / Engineering
Offerte & Pricing	Struttura a Capitoli	Suddivisione delle offerte in capitoli gestiti come sottodistinte personalizzabili.	Commerciali
Offerte & Pricing	Pricing centralizzato e override	Prezzi sincronizzati da Ad-Hoc con possibilità di inserimento manuale per voci speciali o su misura.	Commerciali, Admin
Offerte & Pricing	Scontistica multi-livello	Calcolo sconti combinabili: per riga prodotto, per capitolo, globale (esclusa MDO), globale (inclusa MDO).	Commerciali
Offerte & Pricing	Calcoli tecnici ed economici	Calcolo automatico di potenza elettrica, consumo aria compressa, pesi, volumi e stima costi montaggio/trasporto.	Commerciali
Offerte & Pricing	Vincoli Incoterms e Paesi	Selezione assistita Incoterms (con vincoli UE/extra-UE); adattamento automatico a valuta, lingua e normative (es. UL-CSA).	Commerciali, Logistica
Offerte & Pricing	Alert informativi	Notifiche a video non bloccanti per segnalare scelte non conformi alle specifiche standard d'impianto.	Commerciali
Offerte & Pricing	Versioning e clonazione	Storico revisioni tracciato (Rev.01, Rev.02...) e funzione di duplicazione rapida di offerte esistenti.	Commerciali
Ordini & Workflow	Conversione e congelamento	Blocco del contratto commerciale alla conferma cliente e assegnazione codifica causale ordine.	Commerciali, Amministrazione
Ordini & Workflow	Notifiche multi-reparto	Alert push automatici inviati a Amministrazione, PM, Logistica e Produzione all'apertura dell'ordine.	Sistema (Cross-funzionale)
Ordini & Workflow	Esportazione verso ERP	Invio automatizzato dell'ordine confermato ad Ad-Hoc per avviare la fatturazione.	Amministrazione
Revisione Tecnica	Contratto tecnico parallelo	Gestione di una variante tecnica interna (senza valore legale) gestita dal PM con costificazione separata.	Project Management
Revisione Tecnica	Workflow decisionale modifiche	Bivio post-revisione: approvazione commerciale con nuovo contratto/RDA (impatto alto) o gestione interna (impatto basso).	PM, Commerciale
Stampe & Documenti	Layout di stampa per reparto	Generazione PDF personalizzati per Cliente, Amministrazione e Ufficio Tecnico.	Tutti i reparti
Localizzazione	Supporto multilingua	Interfaccia localizzata e traduzione automatica/manuale delle descrizioni contrattuali.	Commerciali, Agenti

Figura 4.1: Requisiti funzionali raccolti.

4.2 Requisiti non funzionali

Come sopra elenco i requisiti non funzionali emersi. All'interno della tabella seguente saranno anche presenti le priorità assegnate a ciascun Item.

Categoria	Requisito Non Funzionale	Priorità	Specifica Tecnica / Obiettivo
Affidabilità	Eliminazione crash e blocchi	Critica	Stabilità operativa durante inserimento massivo righe, calcoli e stampa, azzerando i blocchi tipici di Access.
Interoperabilità	Connessione nativa ERP	Critica	Integrazione bidirezionale diretta via query SQL Server con Ad-Hoc (anagrafiche, listini ed export ordini).
Disponibilità	Architettura offline (LocalDB)	Alta	Continuità operativa garantita anche in assenza di rete o VPN tramite istanza locale SQL Server LocalDB.
Integrità Dati	Sincronizzazione automatica	Alta	Motore di sync bidirezionale tra LocalDB e SQL Server aziendale con logica di risoluzione conflitti.
Resilienza	Backup automatici in VPN	Alta	Salvataggio automatico del database locale durante le connessioni VPN per prevenire perdite o corruzioni dati.
Integrità Dati	Unificazione codici prodotto	Alta	Coerenza assoluta delle codifiche articolo tra Configuratore, Ad-Hoc ERP e Fusion PLM senza disallineamenti.
Usabilità	Workflow guidati (Wizard)	Alta	Percorsi passo-passo che guidano la compilazione per azzerare omissioni o errori di configurazione commerciale.
Usabilità	Viste orientate al ruolo (RBAC)	Alta	Maschere e sezioni della UI mostrate dinamicamente in base alle prerogative del ruolo utente loggato.
Portabilità	Framework moderno	Media	Architettura per ambiente desktop con predisposizione a target mobile.
Prestazioni	Calcolo istantaneo in-memory	Media	Elaborazione immediata a runtime di formule per potenze, consumi d'aria, pesi, volumi e stime logistiche.
Qualità Processo	Area di staging ordini	Media	Area di transito temporanea per la revisione amministrativa delle conferme d'ordine prima del caricamento contabile.
Evolutività	Predisposizione CAD / PLM	Bassa	Architettura aperta al recupero di disegni tecnici e visualizzazione di modelli 3D direttamente da Fusion.

Figura 4.2: Requisiti non funzionali raccolti.

Capitolo 5

Progettazione dell'architettura

5.1 Preambolo sulle scelte architetturali

Le scelte architetturali sono state dettate principalmente dalle tecnologie già in essere aziendali, e in seconda battuta dalle richieste avanzate negli incontri. Prendendo per assodato l'utilizzo di SQLServer si sono cercati dei framework per l'interazione con il database che potessero supportare anche un database locale come SQLite[SQL]. La scelta è ricaduta sull'architettura .NET[.NE] su base "C#" e, in quanto dell'ecosistema Microsoft, è perfettamente integrata col SQL Server. Tale framework supporta il multi piattaforma ed ha una Community che lo aggiorna continuamente. Questi due punti sono fondamentali per far fronte ai futuri debiti tecnici. Selezionato quest'ultimo la scelta della View è ricaduta sempre in un framework .NET ovvero MAUI[MAU]. MAUI è nativamente multipiattaforma in quanto, in fase di compilazione, genera i nativi sia per desktop che mobile.

5.2 Deployment

Il sistema è stato progettato per essere eseguito in locale e mediante connessione transmission control protocol (TCP) si interroga il database. All'interno dell'azienda è presente un solo dominio **AD!** (**AD!**) a internet protocol (IP) statici, questa caratteristica facilita la connessione al database se si è in sede. Chi accede esternamente, mediante VPN, può stabilire una connessione rientrando nel domi-

nio come sopra. In assenza di connessione, verificata tramite ping, si accede alla tabelle locali che una volta ristabilita la connessione vengono sincronizzate.

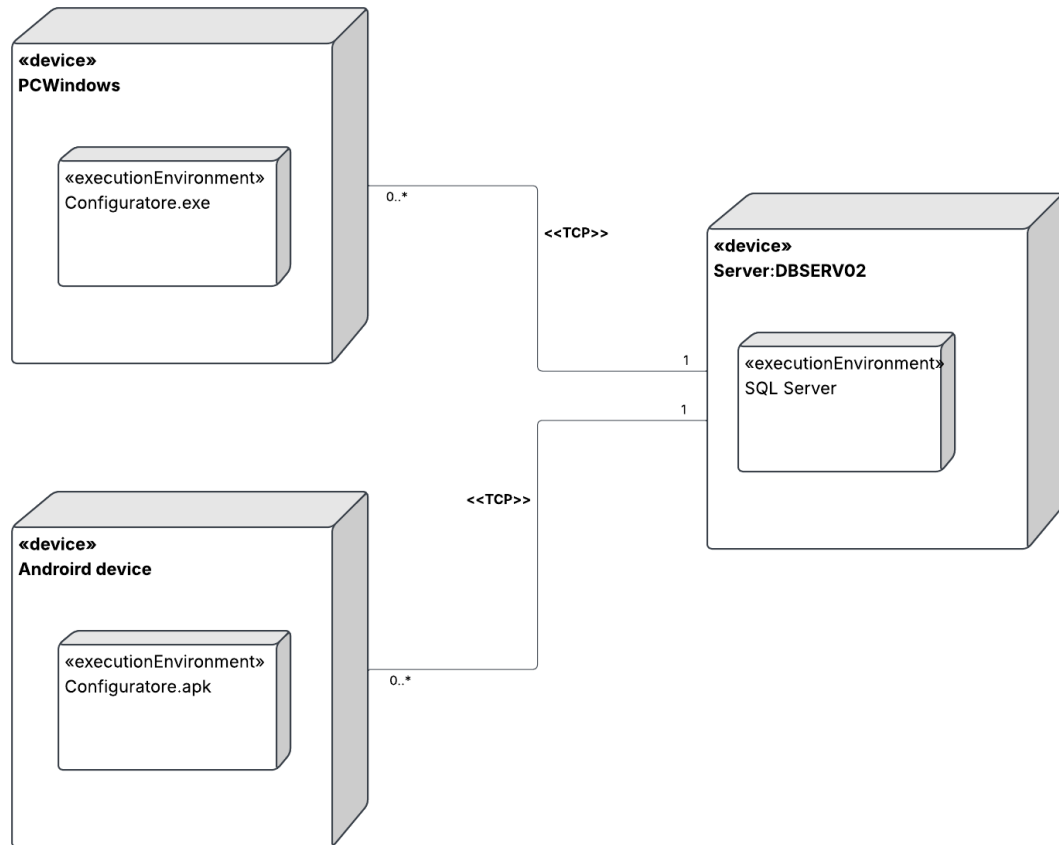


Figura 5.1: Diagramma dei deployment

5.3 Package

Lo sviluppo dell'applicativo ha richiesto una divisione del progetto in tre Package con compiti ben distinti.

5.3.1 Interfaccia al database

Il package di interfaccia al database ha il compito di gestire tutti gli aspetti legati a quest'ultimo. Il package si occupa di:

- Gestire connessione al database.
- Raccogliere e mappare in file neutri le entity corrispondenti alle tabelle database.
- Implementare tutte le operazioni "CRUD".
- Testare mediante classi di test le operazioni precedenti. Questa suite di test è risultata fondamentale per far fronte all'evoluzione del database nel corso dell'implementazione.
- Esporre al di fuori del package solo file neutri che non possano agire direttamente sul database.

Questo modulo si appoggia a sua volta al pacchetto NuGet **MEF! (MEF!)**[\[Micc\]](#), per quanto riguarda l'interazione col database, ed il package "Mapster"[\[Opeb\]](#) per la mappatura delle entity in data transfer object (DTO).

5.3.2 Libreria di componenti View

Si è scelto di racchiudere tutte le factory e le costanti relativi alla view in un package dedicato. Questa scelta è stata detta dalla possibilità di riutilizzare questo componente in moduli o programmi alternativi. La libreria si appoggia a sua volta al package NutGet "Community Toolkit Maui" che racchiude tutte le funzioni per disegnanre i compoenenti view. Normalmente le interfacce MAUI sono scritte mediante codice Markdown, ma attraverso questo package si è potuto scriverle in "C#".

5.3.3 Modulo Main Configuratore

Questo package è il principale che utilizza al suo interno i precedenti. In questo package vengono gestiti tutti gli aspetti relativi alla view come:

- Creazione interfacce
- Gestione delle interfacce mediante ViewModel che hanno il compito di aggiornarle in risposta ad eventi.
- Gestione della navigazione e dei permessi, escludendo dalla view ciò che non può essere fatto.
- Mediante la classe MauiProgram.cs gestione della Dependency Injection.
- In fase di avvio scarica in cache dati dal database fissi, come tabelle di lookup.

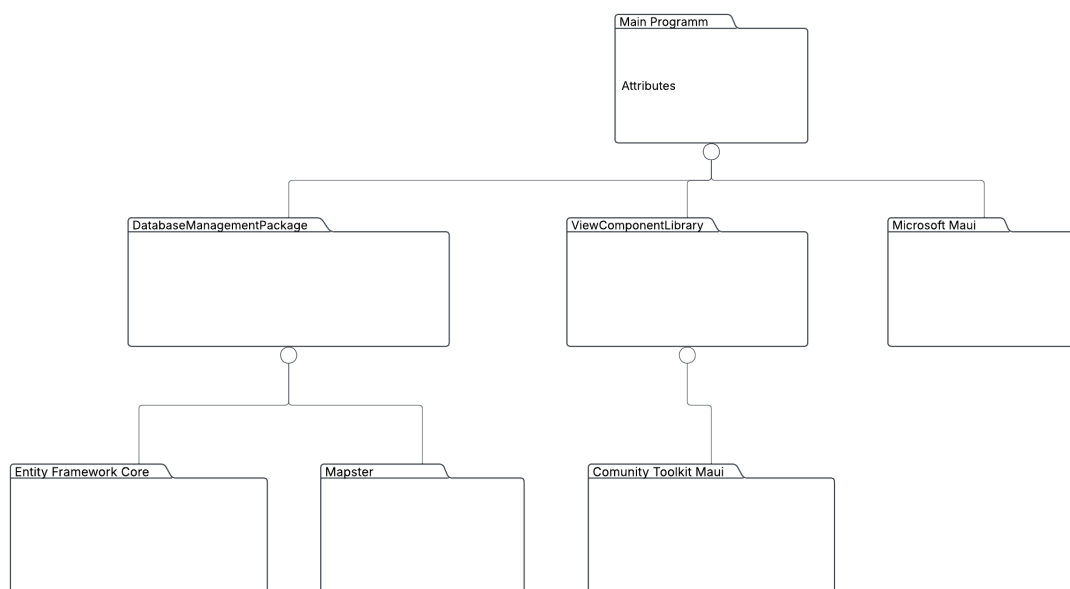


Figura 5.2: Diagramma dei package

5.4 Flussi di lavoro

Osservando i requisiti si è notato che il flusso di un contratto andava rivisto. L’inserimento della revisione tecnica ha permesso una semplificazione dei flussi a valle. Determinare tali flussi è fondamentale nel progettare gli automatismi dovuti alle notifiche intra reparto.

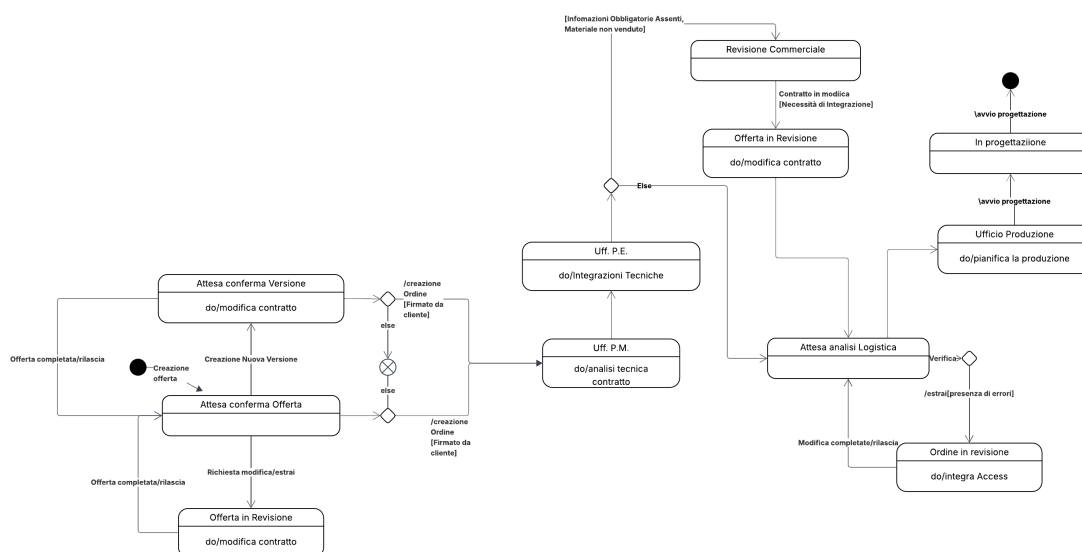


Figura 5.3: Nuovo diagramma degli stati di un contratto

Capitolo 6

Scelte progettuali

6.1 Scelta modello di sviluppo

La scelta del modello di progettazione è ricaduta in un modello agile. Questa scelta è stata dettata principalmente dalla necessità di avere in breve tempo un prototipo da testare con il richiedente. La progettazione è stata supportata dalla creazione di Unit Test. I test sono stati fondamentali per poter validare il codice, in particolare a fronte di aggiornamenti della struttura database. Nel corso dell'implementazione, soprattutto della view, si è spesso seguito un pattern tipico del modello ovvero:

1. Creazione del prototipo della funzione.
2. Creazione della view di riferimento.
3. Test e validazione da parte di un utente scelto.
4. Rifattorizzazione codice se necessario.

La rifattorizzazione del codice è stata affrontata con regolarità per seguire il principio *don't repeat yourself* (DRY), favorendo aggiornamenti più agili a fronte di manutenzione di parti di codice.

6.2 Suddivisione e contenuto dei rilasci

La definizione delle tempistiche di rilascio è stata supportata da un figura di Project Management messi a disposizione. Nel primo rilascio concordato sono compresi solo i requisiti che potessero almeno emulare gli attuali. In questo rilascio, con consegna stimata gennaio 2027, sono presenti le seguenti funzionalità:

- Gestione utenti e permessi.
- Anagrafica clienti.
- Creazione delle offerte.
- Versioning offerte.
- Creazione ordini.
- Import ed export dati, da e verso altri database.
- Creazione reportistica.
- Gestione multilingua interfaccia applicativa.
- Prototipo mobile.
- Modulo di configurazione prodotto.

Si è scelto di non includere la funzionalità offline in quanto essa necessita di un database centrale stabile. La funzione offline verrà implementata nel secondo rilascio quando il software sarà in produzione. Nel secondo rilascio saranno presenti quindi;

- Gestione revisione tecnica e differenze con contratto firmato.
- Inserimento calcoli automatici sulla base delle caratteristiche del venduto (ES:potenza elettrica totale assorbita).
- Gestione multi-valuta per ordini internazionali.
- Apertura disegni .pdf dal server del PLM.

- Report avanzati.
- Implementazione database locale con logiche di sincronizzazione.

Le tempistiche di questo rilascio saranno stabite al termine dei test del primo.

6.3 Scelta Framework ORM

All'interno della sezione 5 ho citato la scelta del framework Entity Framework Core (EF Core). Questo framework ha permesso di mappare le tabelle mediante la tecnica Object-Relational Mapping (ORM). Questa tecnica consiste nel mappare ogn tabella del database in una classe e gestire tutti i legami mediante una classe context. ORM ha permesso, mediante script powershell e connessione al database, di ricreare in modo dinamico tutte le entity a fronte di cambiamenti del database. Tale approccio può essere definito database first, in quanto la progettazione avviene principalmente sul database, il software successivamente. Le query mediante il framework EF Core sono state scritte tramite language integrated query (LINQ) che ha permesso di scriverle in linguaggio "C#". Queste funzioni sono inoltre utilizzabili anche con SQLite in ottica di offline. All'interno degli aspetti negativi era presente la lentezza in grandi volumi dovuti alla traduzione del codice in query. Si è valutato, in caso si presentino di ottimizzare il database mediante indici in prima battuta e successivamente ottimizzando le singole query.

Listing 6.1: Esempio di query LINQ

```
1 public class Example{
2     public async Task<ClientiDestinazioniDTO> GetLocationDetailsById(int?
        idLocation)
3     {
4         ClientiDestinazioni? location = await _db.ClientiDestinazioni
            .Where(l => l.CdeId == idLocation)
5             .Include(l => l.CdeNazioneNavigation)
6             .Include(l => l.CdeTipoNavigation)
7             .Include(l => l.CdeClienteNavigation)
8             .Include(l => l.CdeNormaElettricaNavigation)
9             .Include(l => l.CdeZonaNavigation)
10            .AsNoTracking()
11            .FirstOrDefaultAsync();
12
13         return _mapper.Map<ClientiDestinazioniDTO>(location!);
14     }
15 }
```

6.4 Scelta Framework grafico

La scelta del framework relativo alla view utente era, come da requisiti, vincolata alla portabilità ed all'esecuzione offline. La decisione era quindi ricaduta su due scelte principali ovvero:

- Front end web based, con utilizzo di html, css, javascript e mediante api interrogare il database centrale.
- Utilizzo del Framework MAUI, nativamente cross-platform ed integrato perfettamente con Backend.

La scelta dell'utilizzare una interfaccia web, utile per avere view che si adattano a dispositivi diversi, è stata bocciata principalmente per due motivi. La difficoltà principale era l'utilizzo in locale. In assenza di connessione tutti i file ed il database locale devono essere presenti sul dispositivo stesso. Questa caratteristica rende difficile gestire gli aggiornamenti. Il secondo motivo è l'utilizzo di più linguaggi di programmazione che non massimizzano l'esperienza acquisita con altri moduli. In ottica di ridurre la tempistica nell'avere un primo test con gli utenti essa era considerata un malus. Il framework MAUI di .NET sopperisce ad entrambi i problemi. Il programma è fisicamente installato sui dispositivi ed il loro aggiornamento può essere gestito centralmente all'interno di uno store Microsoft. Essendo esso in base .NET e, mediante il framework "Community Toolkit Maui", le interfacce sono scritte in base "C#".

6.5 Scelta del pattern di progettazione

Insieme all'utilizzo di MAUI è suggerito da Microsoft il pattern Model View View-Model (MVVM). Il modello MVVM consente di separare in modo pulito la logica di business e presentazione di un'applicazione dalla relativa interfaccia utente. La gestione di una separazione netta tra la logica dell'applicazione e l'interfaccia utente consente di risolvere numerosi problemi di sviluppo e semplifica il test, la gestione e l'evoluzione di un'applicazione. Può anche migliorare significativamente le opportunità di riutilizzo del codice e consente agli sviluppatori e ai progettisti dell'interfaccia utente di collaborare più facilmente durante lo sviluppo delle

rispettive parti di un'app.[Micb] Il modello MVVM è composto da tre elementi fondamentali che interagiscono come segue: Questa modello, separando la logica

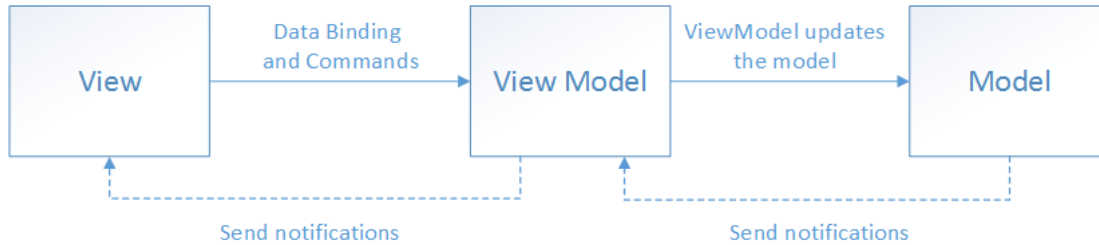


Figura 6.1: Diagramma funzionamento MVVM

della view da quella di business, ha permesso di prototipare le interfacce nei primi rilasci e in futuro riprogettarle migliorando l'accessibilità e User Experience (UX).

Capitolo 7

Progettazione del database

7.1 Esportazione Vecchio Db

7.2 Creazione regole di denominazione

7.3 Utilizzo di SSIS nel processo di migrazione

7.4 Ristrutturazione database

7.4.1 Eliminazione ridondanze e normalizzazione

7.4.2 Creazione tabelle di lookup

7.4.3 Aggiunta tabelle non presenti

7.4.4 Creazione tabelle di Back Up

7.4.5 Scelta di strutture Avanzate come Triggers e Stored Procedure

7.5 Schema logico finale

7.6 Popolamento database

³⁰7.7 Dati fissi

7.8 Dati fissi da vagliare

7.9 Dati provenienti da altri DB

7.9.1 Scelta dei dati

Capitolo 8

Creazione Model

8.1 Mapping delle classi

8.2 Struttura del package

8.2.1 Classi di Test

8.2.2 Repository

8.2.3 Classi ed Interfacce esposte

Capitolo 9

Creazione View

9.1 Utilizzo di Figma per disegno interfacce

9.2 Creazione delle Factory per le View

9.3 Scelta delle classi di viewmodel

Capitolo 10

Modulo controller

10.1 Utilizzo della Dependency Injection

10.1.1 Utilizzo di multi Threading per la gestione dei dati

10.2 Scelta gestione transazione e dei commit

10.3 Utilizzo della Cache in memoria per la gestione di dati fissi

Capitolo 11

Modulo configurazione prodotto

11.1 Scopo del modulo

11.2 Focus group individuazione metodologia

11.3 Studio sulla guida alla configurazione

11.4 Query utilizzate per la ricerca

Bibliografia

- [Das] Dassault Systèmes. Solidworks. <https://www.solidworks.com/>.
- [Fig] Figma. Figma. <https://www.figma.com/>.
- [MAU] .NET MAUI. .net maui. <https://dotnet.microsoft.com/en-us/learn/maui>.
- [Mica] Microsoft. Entity framework core (ef core). <https://learn.microsoft.com/it-it/ef/core/>.
- [Micb] Microsoft. maui.mvvm. <https://learn.microsoft.com/it-it/dotnet/architecture/maui/mvvm>.
- [Micc] Microsoft. Microsoft entity framework. <https://learn.microsoft.com/en-us/ef/>.
- [Micd] Microsoft. Sql server. <https://www.microsoft.com/sql-server>.
- [Mice] Microsoft. Sql server integration services. <https://www.microsoft.com/sql-server/integration-services>.
- [Micf] Microsoft. Sql server migration assistant (ssma). <https://www.microsoft.com/sql-server/migration-assistant>.
- [.NE] .NET. .net. <https://dotnet.microsoft.com/>.
- [Opea] OpenSource. Communitytoolkit.maui. <https://github.com/CommunityToolkit/Maui>.
- [Opeb] OpenSource. Mapster. <https://github.com/MapsterMapper/Mapster>.

- [Rul] Rule Designer. Fusion. <https://www.rulesdesigner.com/it/>.
- [S.p] Cepi S.p.A. Azienda di impianti di stoccaggio. Azienda di Impianti di stoccaggio, <https://www.cepisilos.com/it/>.
- [SQL] SQLite. Sqlite. Lightweight database engine, <https://www.sqlite.org/>.
- [Zuc] Zucchetti. Ad-hoc. <https://www.zucchetti.com/it-it/soluzioni/gestionali>.

Acknowledgements

Optional. Max 1 page.